人工智能导论　第二次作业

学号：63026201　　姓名：吴雨鸥　　题目：用 Lean 形式证明 Euclid 定理

一、问题描述

Euclid 定理：对任意正整数 n，都存在素数 $p > n$。即素数有无穷多个。本作业设计一个 **能自主调用 Lean 编译器验证的 Agent**，把这条定理完整形式化，并以 `lake build` 零错误作为唯一通过判据。

形式化目标（比题面更强，覆盖全部自然数）：

```
theorem exists_prime_gt (n : ℕ) : ∃ p, n < p ∧ Nat.Prime p
```

二、证明思路（Euclid 阶乘构造）

1. 取 $m = n! + 1$；由 $n! > 0$ 得 $1 < m$，故 $m \neq 1$。
2. 令 $p = m$.minFac（m 的最小因子），则 p 是素数且 $p \mid m$。
3. 反设 $p \leq n$：因 p 是 $\leq n$ 的正数，故 $p \mid n!$。
4. 又 $p \mid n! + 1$，两者相消得 $p \mid 1$。
5. 素数不整除 1（`Nat.Prime.not_dvd_one`），矛盾 $\Rightarrow n < p$。

关键：整除性在「$n!$」与「$n!+1$」之间架起矛盾，这正是 Euclid 反证法的精髓。

三、Lean 形式化代码（已编译通过）

```
-- mathlib v4.35 有模块系统: module + public 三者缺一不可
module
public import Mathlib
public section

namespace Euclid

/-- 主定理: Euclid 定理, 阶乘构造法。 -/
theorem exists_prime_gt (n : ℕ) : ∃ p, n < p ∧ Nat.Prime p := by
  let m := n.factorial + 1
  have hm_gt_one : 1 < m := by
    simpa [m] using Nat.succ_lt_succ (Nat.factorial_pos n)
  have hm_ne_one : m ≠ 1 := ne_of_gt hm_gt_one
  let p := m.minFac
  have hp_prime : Nat.Prime p := by
    simpa [p] using Nat.minFac_prime hm_ne_one
  have hp_dvd_m : p ∣ m := by
    simpa [p] using Nat.minFac_dvd m
  have hnp : n < p := by
    by_contra h
    have hp_le_n : p ≤ n := le_of_not_gt h
    have hp_dvd_fact : p ∣ n.factorial :=
      Nat.dvd_factorial hp_prime.pos hp_le_n
    have hp_dvd_m' : p ∣ n.factorial + 1 := by simpa [m] using hp_dvd_m
    have hp_dvd_one : p ∣ 1 := (Nat.dvd_add_iff_right hp_dvd_fact).2 hp_dvd_m'
    exact hp_prime.not_dvd_one hp_dvd_one
  exact ⟨p, hnp, hp_prime⟩

/-- 题面形式: 正整数 n 之上必有素数 -/
theorem exists_prime_gt_pos (n : ℕ) (_hn : 0 < n) : ∃ p, n < p ∧ Nat.Prime p :
  exists_prime_gt n

end Euclid
```

四、依赖的 mathlib 定理

定理	作用
`Nat.factorial_pos`	$0 < n!$，推出 $m \neq 1$
`Nat.minFac_prime`	最小因子是素数
`Nat.minFac_dvd`	最小因子整除 m
`Nat.dvd_factorial`	$p \leq n \Rightarrow p \mid n!$
`Nat.dvd_add_iff_right`	由 $p \mid n!$、$p \mid n!+1$ 得 $p \mid 1$
`Nat.Prime.not_dvd_one`	素数不整除 1 ⇒ 矛盾

五、Agent 设计：验证闭环

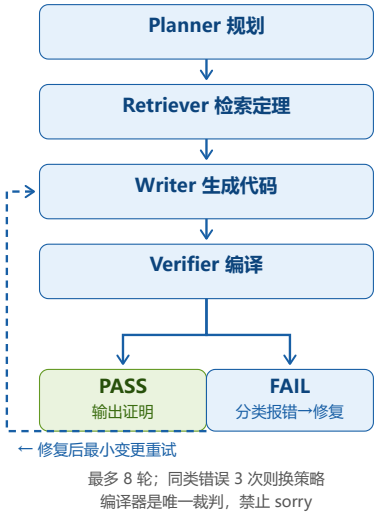


图 1　证明 Agent 的「规划—验证—反思」闭环

工具集：`lake build`（验证）、`#check`（探活）、grep mathlib 源码（检索定理）、补丁与快照（最小变更）。

六、真机验证结果

```
$ lake build Euclid
Build completed successfully (9047 jobs).

$ #check
Euclid.exists_prime_gt (n : ℕ) :
    ∃ p, n < p ∧ Nat.Prime p

$ #print axioms Euclid.exists_prime_gt
depends on axioms:
    [propext, Classical.choice, Quot.sound]
```

环境：Lean 4.35.0 + mathlib v4.35.0，Windows

七、踩坑与收获

① 模块系统：只写 `module` 不够，声明要进 `public section`，签名里的常量还要 `public import`，否则下游 import 查不到。
② 目标登记：`lake build Euclid` 需在 lakefile 里加 `lean_lib`。
③ 不可零依赖：`Nat.Prime`、`Nat.factorial` 只在 mathlib 中，核心库没有。
结论：形式化证明的价值在于「机器可判定」——Agent 的每一次修改都必须 交回编译器裁决，这正是符号 AI 区别于生成式 AI 的地方。